BDIP: An Efficient Big Data-Driven Information Processing Framework and Its Application in DDoS Attack Detection

Qiyuan Fan, Xue Li, Puming Wang, Xin Jin, Shaowen Yao, Shengfa Miao, *Member, IEEE*, Sizhang Li, Min An, and Jing Xu

***Abstract*—With the rapid advancement of 5G communication technology in the era of big data, massive terminal devices connected to the Internet have dramatically increased the scale of network, generating a large amount of high-dimensional and heterogeneous information. This not only enhances the difficulty of information processing in the network, but also poses a severe challenge to data storage and calculation, which has become a big data problem to be solved urgently. To cope with it, this paper proposes an efficient information processing framework and applies it to Distributed Denial of Service (DDoS) attack detection. Overall, three major highlights are made: (i) Tensor is used to represent multi-modal information in large-scale networks; (ii) A novel denoising algorithm based on tensor train(TT) decomposition is proposed, focused on optimizing both computation and correlation; (iii) A big data-driven information processing framework is developed, which includes information preprocessing, denoising and classification. Results in case study indicate that the framework can achieve an accuracy of 99.19%, all while maintaining the great storage advantage, well speedup ratio and strong computing capabilities under the same computational complexity. It can also be generalized to other network data processing scenarios.**

***Index Terms*—Tensor networks, TT decomposition, storage efficiency, information processing, DDoS attack detection, machine learning.**

I. Introduction

IN RECENT years, 5G communication technology has provided mobile devices with on demand, flexible, manageable system resources and services, greatly facilitating people's lives. However, this has also led to a significant increase in the scale of information networks and a more complex network structure, which not only brings challenges for information processing but also creates many opportunities for network attacks [1].

Received 20 February 2024; revised 3 July 2024; accepted 8 September 2024. Date of publication 20 September 2024; date of current version 14 March 2025. This work was supported in part by the National Nature Science Foundation of China under Project 62166047; in part by the Yunnan International Joint Laboratory of Natural Rubber Intelligent Monitor and Digital Applications under Grant 202403AP140001; in part by the Xingdian Talent Support Program under Grant YNWR-QNBJ-2019-270; and in part by The 15th Graduate Research Innovation Project of Yunnan University under Grant KC-23234593. The associate editor coordinating the review of this article and approving it for publication was Y.-D. Lin. *(Corresponding author: Puming Wang.)*

Qiyuan Fan and Jing Xu are with the School of Software, Yunnan University, Kunming, China

Xue Li is with the School of Electronic Information Engineering, Henan Institute of Technology, Xinxiang 453002, China.

Puming Wang, Xin Jin, Shaowen Yao, Shengfa Miao, Sizhang Li, and Min An are with the School of Software, Yunnan University, Kunming 650091, China (e-mail: pumingwang@gmail.com).

Digital Object Identifier 10.1109/TNSM.2024.3464729

Common network attacks include Distributed Denial of Service (DDoS), wrapping attack, malware injection attack, launch malicious Virtual Machine (VM) and VM Escape. It is DDoS that has become one of the most notorious attack among them, since its purpose is to first consume system resources, then cause network overloading, and eventually stop providing normal services. To make matters worse, the number of DDoS attacks has increased sharply over the past few years [2], resulting in a great deal of important information cannot be processed timely. In addition, individuals and countries are increasingly dependent on the Internet and network computing, hence, it is urgent to find out an effective information processing framework to detect DDoS attacks.

Actually, in the era of big data, how to represent information in the huge and complex network has become the first major challenge in detecting DDoS attacks, and it has been proved that tensor can represent large-scale high-dimensional data well and fuse heterogeneous data effectively [3]. In this paper, tensor is used to represent datasets. After that, when we train the processing model, datasets are always noise-free by default, but collecting and labelling data manually in reality will inevitably introduce noise into datasets, so that the quality of datasets will be reduced and negatively affecting the results [4]. Thus, efficient denoising for datasets is a crucial step. Data processing method based on tensor train (TT) decomposition has the characteristics of easy dimension reduction, strong computing capabilities and great storage advantage, making it an ideal choice for denoising large-scale datasets. However, traditional TT decomposition is computation-intensive and overlooks the correlation between data, either of which can affect the model to achieve the optimal result. To address this issue, we propose an improved TT decomposition for denoising, which can solve the above problems by tensor low-rank approximation and reuse, respectively. Additionally, researchers have proposed many classification models and the model based on machine learning (ML) has become the most popular one because of its good effect and lightweight structure. Thus, Light Gradient Boosting Machine (LightGBM) proposed recently can be a nice choice. Comparing with other models, it can offer lower memory consumption, stronger computing capabilities and the ability to process data in parallel, naturally conforming to the improved TT decomposition and large-scale information processing scenario.

On the whole, the primary highlights can be outlined in the following manner:

- We propose the improved and efficient TT(IETT) decomposition algorithm for data denoising;
- We propose to combine IETT denoising and LightGBM classification in a distributed parallel environment, seamlessly leveraging the strengths of both approaches;
- We propose a big data-driven framework for information processing and apply it to DDoS attack detection.

The remainder of paper is structured as below. Related work is recalled in Section II, and some preliminary knowledge is introduced in the next section. After that, an efficient and improved TT decomposition based framework for information processing is proposed in Section IV. Section V describes the case study by applying the framework to DDoS attack detection. Summary and future work are given in Section VI.

II. Related Work

Detecting DDoS attack detection is a vital application for the information processing framework. Therefore, numerous R&D teams have conducted research in this area, trying to detect and deal with it timely. Broadly, processing frameworks are classified into three main categories: statistic model, ML model, hybrid model.

A. Statistic Model

In terms of traffic characteristics, DDoS attack is different from normal network, and entropy is a statistical method to measure information uncertainty. Therefore, methods based on statistic models can identify possible indicators by analyzing shifts in entropy.

Anderson [5] proposed the intrusion detection as early as 1980. Denning [6] designed the IDS model 6 years later, which laid an important foundation for the subsequent research on information processing, especially the DDoS attack detection. Bhuyan et al. [7] proposed the use of partial rank correlation for detecting high-rate and low-rate DDoS attacks, where results were highly reliant on threshold selection. Çakmakçı et al. [8] created a dictionary that quantifies genuine normal traffic along with its characteristics, and the detection index was determined by calculating the distance between a member of the dictionary distribution and a suspicious vector.

In conclusion, a notable limitation of these models is the requirement to establish a suitable threshold for detection entropy, as simply observing traffic attributes is inadequate to differentiate between normal and abnormal traffic.

B. Machine Learning Model

The core function of ML and deep learning (DL) models is to correctly categorize traffic datasets.

In the field of ML, Barradas et al. [9] proposed FlowLens, a machine learning system for traffic classification in security network applications. It introduces a memory-efficient representation called flow marker. By using a profiler in the control plane, FlowLens generates application-specific markers that optimize resource consumption and classification accuracy. Wang et al. [10] proposed to combine sequential feature selection with Multi Layer Perceptron(MLP) to obtain a feedback mechanism for abnormal traffic detection based on dynamic perception.

In the field of DL, Doriguzzi-Corin et al. [11] tried to enhance the deployment stability. They employed deep CNNs in resource-limited environments, which achieved the efficient information processing. Apruzzese et al. [12] introduced a approach to pragmatic evaluations, suggesting a shift towards assessing the values prioritized by real-world industrial operators who implement intrusion detection systems. It is worth noting that there is currently a lack of empirical data regarding the evaluation of intrusion detection systems.

Generally speaking, most ML based models require large datasets to train, which increases the consumption of computing and storage resources.

C. Hybrid Model

In contrast to traditional information processing algorithms, the hybrid model is able to combine the advantages of multiple algorithms, enabling more effective intrusion detection.

Dehkordi et al. [13] introduces an effective SDN-based DDoS detection method using machine learning and statistical analysis. Results demonstrate its superior in accuracy and false positive rates. Swamy et al. [14] presented Taurus, a data plane architecture employing custom hardware and parallel pattern abstraction for per-packet MapReduce operations. It accelerates machine learning inference and showcases performance benefits in data center networks. Wichtlhuber et al. [15] describes a hybrid system called IXP Scrubber, which detects and filters DDoS traffic at Internet exchange points (IXPs) by constantly learning the DDoS traffic characteristics of neighboring autonomous systems. It uses black hole traffic as training data and combines it with machine learning models for classification, while using rule mining and interpretability techniques to improve interpretability and controllability

Overall, both statistic and machine learning models struggle to effectively manage information in complex networks. While a few hybrid models may yield relatively positive results, they are generally inefficient and consume a sum of storage space. Therefore, to address these issue, an efficient big data-driven hybrid model based on tensor is proposed, offering high detection accuracy and low memory consumption.

III. Preliminaries

Some preliminaries are introduced in the section. Section III-A introduce notations used throughout the following sections. The concept of tensor algebra are introduced in Section III-B, and Section III-C describes the data model.

A. Notations

Based on the established notation conventions [16], we denote lowercase letters, bold lowercase letters, bold uppercase letters and calligraphic uppercase letters as scalars, vectors, matrices and tensors. Moreover, we use $\mathbf{X}^T$ to express the transpose matrix, $\mathbf{X}_{[i,:]}$ to represent all elements in row i, $\mathbf{X}_{[i,j]}$ to denote the element of row i column j. Furthermore, subarry spanning from row i to j is expressed as $\mathbf{X}_{[i:j,:]}$, sets of real numbers are denoted as $\mathbb{R}$, a K-mode tensor is represented

TABLE I
NOTATIONS

Symbol	Description
x	Scalar
$\mathbf{x}$	Vector
$\mathbf{X}$	Matrix
$\mathbf{X}^T$	Transpose of matrix $\mathbf{X}$
$\mathbf{X}_{[i,:]}$	All elements in row i
$\mathbf{X}_{[i,j]}$	The element in row i and column j
$\mathbf{X}_{[i:j,:]}$	Subarray
$\mathcal{X}$	Tensor
$\mathbb{R}$	Real numbers
$\mathcal{X} \in \mathbb{R}^{I_1 \times \cdots \times I_K}$	K-mode tensor
$\mathcal{X}_{[I_1,\cdots,I_{n-1},:,I_{n+1},\cdots,I_N]}$	K-mode fiber
$\mathcal{X}_{[:,\cdots,I_k:I_k+1,\cdots,:]}$	K-dimension subtensor
$\mathbf{X}_{(n)} \mathbb{R}^{(\prod_{i \neq n} N_i) \times I_n}$	Matricization of tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$
$\mathbf{X}_{SS,M}^{(L_M)} \in \mathbb{R}^{M^{sub} \times (N \times L_M)}$	Smoothing matrix of $\mathbf{X} \in \mathbb{R}^{M \times N}$
$\mathcal{X}_{SS,k}^{(L_k)} \in \mathbb{R}^{N_k^{sub} \times L_k \times \prod_{i \neq k} N_i}$	Smoothing tensor of $\mathcal{X} \in \mathbb{R}^{N_1 \times \cdots \times N_K}$
$\odot$	Tensor convolution
$*$	Tensor product

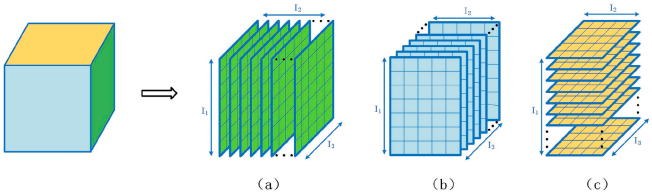


Fig. 1. (a) Lateral slices (b) Frontal slices (c) Horizontal slices.

as $\mathcal{X} \in \mathbb{R}^{I_1 \times \cdots \times I_K}$, containing $N = \prod_{i=1}^{K} I_k$ elements. All mathematical symbols are listed in Table I.

B. Tensor Algebra

According to the relevant knowledge of tensor, the preprocessing and decomposition of tensor can be defined as follows:

Definition 1 (Slices): Slicing tensors means extracting higher-dimensional tensors along a certain dimension to get lower-dimensional tensors. In the case of a three-dimensional tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$, if you let I_1 and I_2 dimensions vary and fix the I_3 dimension, you will get a two-dimensional matrix $\mathbf{X} \in \mathbb{R}^{I_1 \times I_2}$. Hence, $\mathbf{X}_{[:,j,:]}$, $\mathbf{X}_{[:,:,k]}$ and $\mathbf{X}_{[i,:,:]}$ shown in Fig. 1 represent the slices from lateral, frontal and horizontal.

Definition 2 (Matricization): Matrixization is to unfold a tensor along the *k-th* dimension and represent it as a series of matrices. Fig. 2 shows the process of matrixization with a three-dimensional tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$ as an example, and $\mathbf{X}_{(1)} \in \mathbb{R}^{I_1 \times (I_2 \times I_3)}$, $\mathbf{X}_{(2)} \in \mathbb{R}^{I_2 \times (I_3 \times I_1)}$ and $\mathbf{X}_{(3)} \in \mathbb{R}^{I_3 \times (I_1 \times I_2)}$ are the corresponding matrices under three modes.

Definition 3 (Spatial Smoothing): Firstly, matrix $\mathbf{X} \in \mathbb{R}^{P \times Q}$ can be split into L_P submatrices $\mathbf{X}^{(l_p)} \in \mathbb{R}^{P^{sub} \times Q}$ along rows and L_Q submatrices $\mathbf{X}^{(l_q)} \in \mathbb{R}^{Q^{sub} \times P}$ along columns, where $P^{sub} = P - L_P + 1$, $l_p \in \{1, 2, \ldots, L_P\}$ and $Q^{sub} = Q - L_Q + 1$, $l_q \in \{1, 2, \ldots, L_Q\}$. Then, the spatial smoothing preprocessing scheme [17] is applied in each matrix for $p = 1, \ldots, P$ and $q = 1, \ldots, Q$. After that, we can express

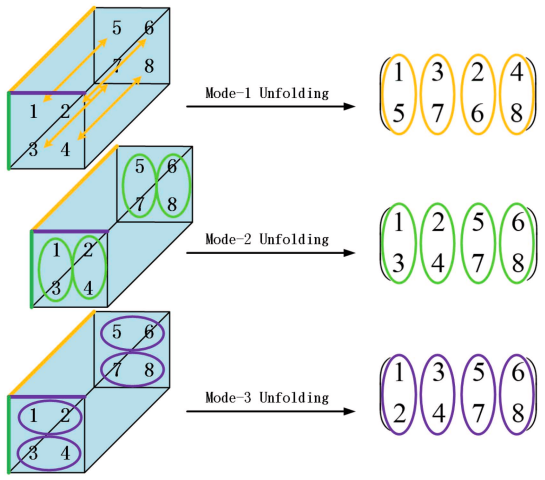


Fig. 2. Matricization of a three-dimensional tensor $\mathcal{X}$.

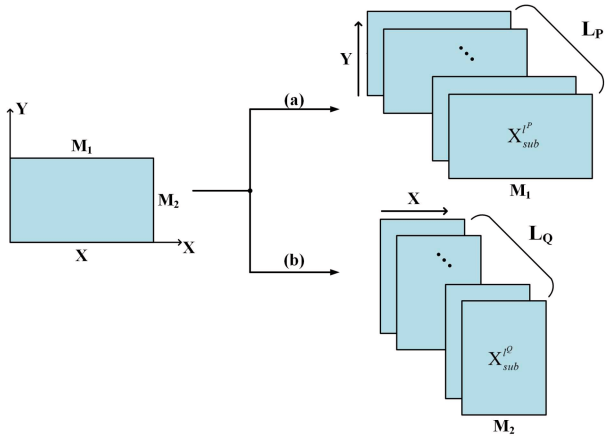


Fig. 3. (a) Smoothing in Y direction, (b) Smoothing in X direction.

the *Pth-mode* and *Qth-mode* smoothing matrix as $\mathbf{X}_{SS,P}^{(L_P)}$ and $\mathbf{X}_{SS,Q}^{(L_Q)}$:

$$\begin{aligned} \mathbf{X}_{SS,P}^{(L_P)} &= \left[\mathbf{X}^{(1)}, \ldots, \mathbf{X}^{(L_P)}\right] \in \mathbb{R}^{P^{sub} \times (Q \times L_P)}, \\ \mathbf{X}_{SS,Q}^{(L_Q)} &= \left[\mathbf{X}^{(1)}, \ldots, \mathbf{X}^{(L_Q)}\right] \in \mathbb{R}^{Q^{sub} \times (P \times L_Q)}. \end{aligned} \tag{1}$$

Based on equation (1), the smoothing process of matrix $\mathbf{X} \in \mathbb{R}^{M_1 \times M_2}$ is illustrated in Fig. 3. Smoothing can be in either the X direction or the Y direction, and $\mathbf{X}_{SS,1}^{(L_1)} = [\mathbf{X}_1^{(1)}, \ldots, \mathbf{X}_1^{(L_1)}] \in \mathbb{R}^{P_1^{sub} \times (P_1 \times L_1)}$ represents the spatial smoothing matrix in *X* direction, $\mathbf{X}_{SS,2}^{(L_2)} = [\mathbf{X}_2^{(1)}, \ldots, \mathbf{X}_2^{(L_2)}] \in \mathbb{R}^{Q_2^{sub} \times (Q_2 \times L_2)}$ represents the spatial smoothing matrix in *Y* direction.

Definition 4 (Tensor Product): The tensor product $\mathcal{Z}$ of three-dimensional tensors $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$ and $\mathcal{Y} \in \mathbb{R}^{I_2 \times I_4 \times I_3}$ is defined as follows,

$$\begin{aligned} \mathcal{Z} &= \mathcal{X} * \mathcal{Y} \in \mathbb{R}^{n_1 \times n_4 \times n_3}, \\ \mathcal{Z}_{[i,j,:]} &= \sum_{k=1}^{n_2} \mathcal{X}_{[i,k,:]} \odot \mathcal{Y}_{[k,j,:]}. \end{aligned} \tag{2}$$

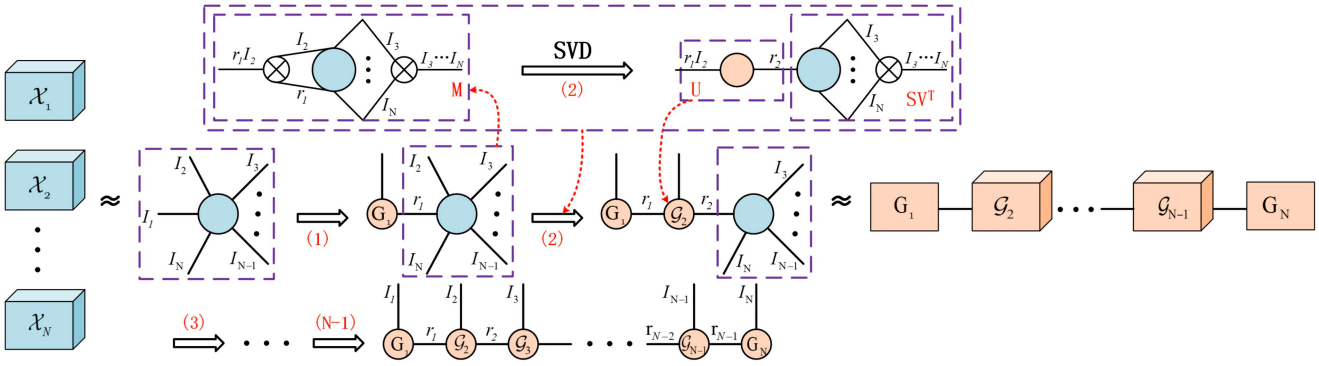


Fig. 4. TT decomposition of an N-dimensional tensor [18]. Firstly, the original tensor is transformed into a matrix. Subsequently, the SVD algorithm is applied to derive the left singular matrix U, the right singular matrix V_T, and the singular value matrix S. This process is carried out in a loop, and finally all the core tensors $G_1, G_2, \ldots, G_N$ are obtained.

Definition 5 (Tensor Train Decomposition): It is the TT decomposition [19] that has become one of the most representative tensor decomposition algorithm. On account of its unique form of data storage, it has been widely used in clustering, denoising and prediction.

Fig. 4 illustrates the process of TT decomposition. Its main task is to decompose an N-dimensional tensor into $N-2$ three-dimensional tensors located at intermediate and two matrices located at both sides. Thus, the TT decomposition of $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_n \times \cdots I_N}$ can be written as follows,

$$\mathcal{X}(i_1, \ldots, i_n, \ldots, i_N) = \mathcal{G}_1(1, i_1, :)\mathcal{G}_2(2, i_2, :) \cdots \mathcal{G}_n(:, i_n, :) \\ , \ldots \mathcal{G}_N(:, i_N, 1), \tag{3}$$

where $\mathcal{G}_n \in \mathbb{R}^{X_{n-1} \times I_n \times X_n}$ is the core tensor and $\mathcal{G}_1 \in \mathbb{R}^{1 \times I_1 \times X_1}$, $\mathcal{G}_N \in \mathbb{R}^{X_N - 1 \times I_N \times 1}$ are the head and tail matrix.

Besides, this process can also be obtained by $N-1$ times singular value decomposition (SVD) [18]. So the first dimension of $\mathcal{X}$ can also be represented in the following manner,

$$\mathcal{X} = \mathcal{U}_1 * \mathcal{S}_1 * \mathcal{V}_1^T + E_1, \tag{4}$$

where $\mathcal{U}_1$ and $\mathcal{V}_1$ are orthogonal tensors, $\mathcal{S}_1$ is the diagonal tensor, E_1 consists of singular values from other dimensions. To relate SVD and TT decomposition, $\mathcal{S}_1 * \mathcal{V}_1^T$ is regarded as $\overline{\mathcal{M}}_1$ for better representing the correlation of TT cores. Therefore, combining with equation (3) and Fig. 5, equation (4) is able to reformulate as follows,

$$\mathcal{X} = \sum_{t_1=1}^{T_1} \mathcal{G}_1(1, i_1, t_1)\overline{\mathcal{M}}_1(\overline{t_1 i_2}, i_3, \ldots, i_N) + E_1, \tag{5}$$

where T_1 denotes the rank of first TT core, $\mathcal{G}_1(1, :, :) = \mathcal{U}_1$, $\overline{t_1 i_2}$ means concatenating the t_1 and i_2 of $\overline{\mathcal{M}}_1$.

C. Data Model

With the blooming development of 5G communication technology and the Internet, data in the network can presents the following characteristics that need to be noted in intrusion detection:

- Large-Scale: In the age of Internet, the Web can generate an immense number of gigabytes of traffic data in mere seconds, covering a broad spectrum of everyday activities. Consequently, the detection system must swiftly and precisely handle these vast quantities of information;
- Heterogeneous: Typically, network traffic data is produced and gathered from various locations and origins, manifesting in both structured formats (like databases) and unstructured formats (like videos) [20]. Therefore, the framework must have the ability to identify data of diverse types;
- Multi-Modal: Each instance of traffic data encompasses a range of details, including IP address, time stamp, network protocol, and etc. Thus, it is essential to integrate these into a multi-modal dataset during processing, thereby leveraging the full spectrum of available features.

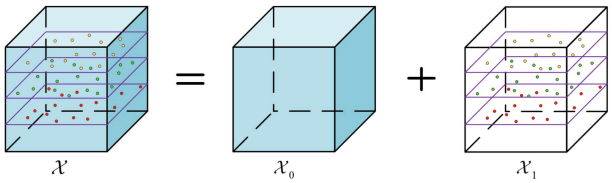


Fig. 5. A three-dimensional tensor data model $\mathcal{X}$. It can be divided into a noise-free tensor $\mathcal{X}_0$ and a noisy tensor $\mathcal{X}_1$.

In order to effectively accommodate these characteristics, tensor, as the multi-dimensional extension of matrix, is employed to represent datasets in our work. Different to matrices, tensors enable a natural utilization of the multi-order properties inherent in network datasets, which helps to capture complex relationships between characteristics and retain the correlation between modals [3]. Therefore, considering the noise that inevitably exists in datasets, the three-dimensional datasets $\mathcal{X}$ shown in Fig. 5 can be divided into a noise-free tensor $\mathcal{X}_0$ and a noisy tensor $\mathcal{X}_1$:

$$\mathcal{X} = \mathcal{X}_0 + \mathcal{X}_1, \tag{6}$$

Thus, we need to reduce the influence of noisy tensor $\mathcal{X}_1$ to noisy tensor $\mathcal{X}_0$ when processing datasets $\mathcal{X}$.

IV. The Proposed Big Data-Driven Information Processing Framework

This section illustrates the proposed big data-driven information processing framework (BDIP) from three aspects.

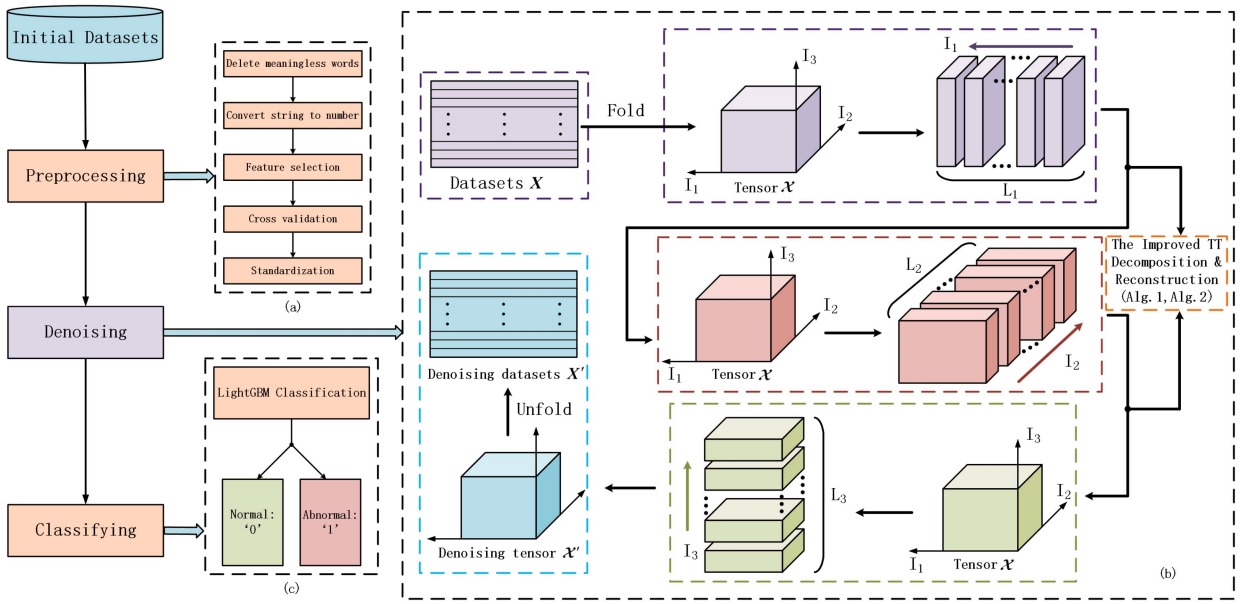


Fig. 6. The proposed DDoS attack detection framework and it can be divided into three parts: (a) Data Preprocessing, (b) Data Denoising, (c) Data Classifying.

As shown in Fig. 6, BDIP consists of three parts. Firstly, Fig. 6(a) introduces data preprocessing, which makes the messy initial datasets computable; then, Fig. 6(b) introduces data denoising, which removes the unnecessary noise to improve the accuracy of processing; finally, Fig. 6(c) introduces data classification, which classifies normal and abnormal traffic information to get the final result.

A. Data Preprocessing

Datasets preprocessing is beneficial for the following denoising and classifying, so a series of methods are introduced in the subsection.

1) Delete Meaningless Words: The first step is to delete some meaningless words so as not to interfere with subsequent works. For example, words like *NAN, INF, IND* will be removed.

2) Convert Strings Into Numeric Values: Then, certain types of strings must be transformed into numeric values. For instance, categorical labels such as 'yes' and 'no' are able to convert into '1' and '0', respectively. This conversion preserves the semantic meaning of the labels, conserves storage space, and enhances computational efficiency.

3) Feature Selection: Considering that there are plenty of features in network datasets, and a portion of them have little impact on the framework, thus we only need to select the more important features which can simplify the calculation without affecting the results.

4) Cross Validation: K-fold cross validation can divide datasets into K blocks, one of which is selected each time as the testing set, and the rest $K-1$ blocks as the training set. After K times, each block is tested once and trained $K-1$ times, and the result will take the mean of K times. In this way, it can effectively avoid overfitting.

Additionally, according to Maranhão et al. [21], the value of K is usually chosen as 5 or 10 to better represent entire datasets. Given the considerable size of the datasets in our study, optimal detection results can still be achieved with a reduced K. Therefore, we opt $K=5$ for a better efficiency of the framework.

5) Standardization: The final step of the data preprocessing is standardization. It can adjust the scale of feature values between [0, 1], while maintaining a portion of the original distribution of the anomalous data caused by DDoS attacks. Hence, this could potentially enhance the precision of the model and expedite its convergence rate. The equation of standardization is shown as follows,

$$nom\left(\mathbf{X}_{[p,q]}\right)=\frac{\mathbf{X}_{[p,q]}-mean\left(\mathbf{X}_{[:,q]}\right)}{std\left(\mathbf{X}_{[:,q]}\right)}, \tag{7}$$

where $std(\mathbf{X}_{[:,q]})$ is the standard deviation, $mean(\mathbf{X}_{[:,q]})$ represents the mean value of $\mathbf{X}_{[:,q]}$.

B. Data Denoising

Some supervised learning algorithms make a specific assumption on noise distribution (such as Gaussian distribution), but generally, they do not assume that the dataset is completely noise-free. Additionally, the origins of data, the collection methodologies, the subjectivity in the labeling process and various other elements may also introduce noise to datasets. Therefore, it becomes imperative to implement denoising techniques. In this subsection, datasets will be denoised by smoothing reconstruction of space and TT decomposition.

1) Tensor Space Smooth and Reconstruction: The tensor space smooth and reconstruction is an expansion of the matrix smoothing technique described in Section III, requiring us to broaden the approach accordingly. Smooth matrix $\mathbf{X}_{SS}^{(L)}$ is assembled from a sequence of submatrices $\mathbf{X}^{(L_i)}$, enabling the extraction of spatial smoothing subtensors $\mathcal{X}^{(l_k)}$. This

is accomplished by applying smoothing to a K-dimensional tensor residing in $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_K}$ specifically along its $K-th$ dimension,

$$\mathcal{X}^{(l_k)} \in \mathbb{R}^{N_k^{sub} \times \prod_{i \neq k} N_i} = \mathcal{X}_{[:,\ldots,l_k:l_k+N_k^{sub},\ldots,:]}, \tag{8}$$

where $\mathcal{X}_{[:,\ldots,l_k:l_k+N_k^{sub},\ldots,:]}$ represents the subtensor intercepted by $\mathcal{X} \in \mathbb{R}^{N_1 \times \cdots \times N_K}$ along the $K-th$ dimension, $l_k \in \{1, 2, \ldots, L_k\}$, $N_k^{sub} = N_k - L_k + 1$, and L_k denotes the number of blocks along the $K-th$ dimension, in which $k \in \{1, 2, \ldots, K\}$. Moreover, if we continuously smooth these subtensors $\mathcal{X}^{(l_k)}$ along the second dimension, we will get a $(K+1)$-dimensional tensor,

$$\mathcal{X}_{SS,k}^{(L_k)} \in \mathbb{R}^{N_k^{sub} \times L_k \times \prod_{i \neq k} N_i} = \left[\mathcal{X}^{(1)}, \ldots, \mathcal{X}^{(L_k)}\right]. \tag{9}$$

Generally speaking, the tensor smooth algorithm strengthens the non-singularity of datasets, providing a theoretical foundation for subsequent decomposition and classification. However, during practical computation, the subtensors that result from smoothing operations conducted in various dimensions may vary in size. Consequently, it is necessary to reconstruct these subtensors to ensure consistency and compatibility. For instance, the K-dimensional tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_K}$ is able to partition into N_K subtensors along its $K-th$ dimension,

$$\mathcal{X} = \begin{bmatrix} \mathcal{X}_{[:,\ldots,1,\ldots,:]} \\ \vdots \\ \mathcal{X}_{[:,\ldots,k,\ldots,:]} \\ \vdots \\ \mathcal{X}_{[:,\ldots,N_K,\ldots,:]} \end{bmatrix} \tag{10}$$

in which $K \in \{1, 2, \ldots, N_K\}$. Furthermore, it is necessary to reconstruct subtensors through the equation:

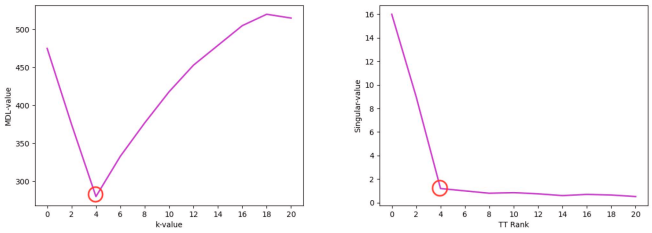
$$\mathcal{X}_{[:,\ldots,k,\ldots,:]} = \frac{1}{l_k} \sum_{i=1}^{l_K} \mathcal{X}_{[:,\ldots,k-i+1,\ldots,:]}^{(i)}, \tag{11}$$

in which l_k denotes the number of smoothing times and $0 \leq k - i + 1 \leq N_K^{sub}$.

To be specific, if tensor $\mathcal{X} \in \mathbb{R}^{3 \times 3 \times 3}$ is smoothed along the second dimension, the smooth tensor $\mathcal{X}_{SS,2}^{(3)} = [\mathcal{X}^{(1)}, \mathcal{X}^{(2)}, \mathcal{X}^{(3)}]$ can be obtained. Based on equation (11), we can denote its reconstruction result $\mathcal{X}'$,

$$\mathcal{X}' = \begin{bmatrix} \mathcal{X}_{[:,1,:]}^{(1)} \\ \frac{1}{2} \times \left(\mathcal{X}_{[:,2,:]}^{(1)} + \mathcal{X}_{[:,1,:]}^{(2)}\right) \\ \frac{1}{3} \times \left(\mathcal{X}_{[:,3,:]}^{(1)} + \mathcal{X}_{[:,2,:]}^{(2)} + \mathcal{X}_{[:,1,:]}^{(3)}\right) \\ \frac{1}{2} \times \left(\mathcal{X}_{[:,3,:]}^{(2)} + \mathcal{X}_{[:,2,:]}^{(3)}\right) \\ \mathcal{X}_{[:,3,:]}^{(3)} \end{bmatrix} \tag{12}$$

2) IETT: Improved and Efficient TT Decomposition: It is TT decomposition that has become a significant method for data denoising. However, when in large-scale, multi-modal information processing scenarios, it will usually produce a amount of computation and it can not maintain the correlation between different modes of data. Hence, we can improve TT decomposition from these two aspects, respectively.



(a) (b)

Fig. 7. Description of MDL principle for TT ranks.

On the one hand, TT rank plays an important role in TT decomposition, low TT ranks can effectively reduce the computation and data feature loss. Thus, we use Minimum Description Length (MDL) principle to get an approximate low TT rank. Based on Wax and Kailath [22], we can get the following equation,

$$\begin{aligned} MDL(t) &= \frac{1}{2} t(2n-t) log L - log \left(\frac{\prod_{k=t+1}^{n} \sigma_k^{1/(n-t)}}{\frac{1}{n-t} \sum_{k=t+1}^{n} \sigma_k} \right)^{L(n-t)}, \\ rank(\mathcal{X}) &= \underset{t}{argmin}(MDL(t)), \end{aligned} \tag{13}$$

where the number of eigenvalues is expressed by n, σ_k denotes the eigenvalue, L represents the length of datasets, $k \in \{1, 2, \ldots, n\}$, $\sigma_1 > \sigma_2 > \cdots > \sigma_n$.

Taking a group of sinusoidal function as an example, $20dB$ zero mean white Gaussian noise is added for denoising by TT decomposition. According to equation (13), Fig. 7(a) depicts the variation of the MDL-value across various k-values, revealing that the MDL attains its lowest value when $k = 4$. Fig. 7(b) illustrates the change in singular values in conjunction with the TT rank, we can still find that singular value reach a high value only when $0 \leq rank \leq 4$. This observation is in agreement with the findings presented in Fig. 7(a). Additionally, singular value can provide useful characteristic information for denoising while TT decomposition can be realized by multiple singular value decomposition, so we truncate the first four sets of singular values in this paper.

On the other hand, traditional TT decomposition can decompose a huge high-dimensional tensor into a series of three-dimensional tensors and matrices, but the correlation between them is not guaranteed. According to Section III, a TT decomposition can be completed by multiple SVDS while SVD have the ability to preserve the correlation between different modes. On this basis, we propose the reuse of tensors. Specifically, after each decomposition, we retain the orthogonal tensor $\mathcal{U}$ as the core tensor, the tensor product of another orthogonal tensor $\mathcal{V}^T$ and the diagonal tensor $\mathcal{S}$ is used as the input for the next decomposition.

Taking the tensor $\mathcal{X} \in \mathbb{R}^{L_1^{(1)} \times L_2^{(1)} \times \cdots \times L_N^{(1)} \times L_1^{(2)} \times L_2^{(2)} \times \cdots \times L_N^{(2)} \times \cdots \times L_1^{(n)} \times L_2^{(n)} \times \cdots \times I_N^{(n)}}$ as an example. Firstly, we use TT decomposition on dimension $L_1^{(1)}, L_2^{(1)}, \ldots, L_N^{(1)}$ to obtain $\mathcal{X}_1$,

$$\mathcal{X}_1 = \mathcal{U}_{(1)} * \mathcal{S}_{(1)} * \mathcal{V}_{(1)}^T, \tag{14}$$

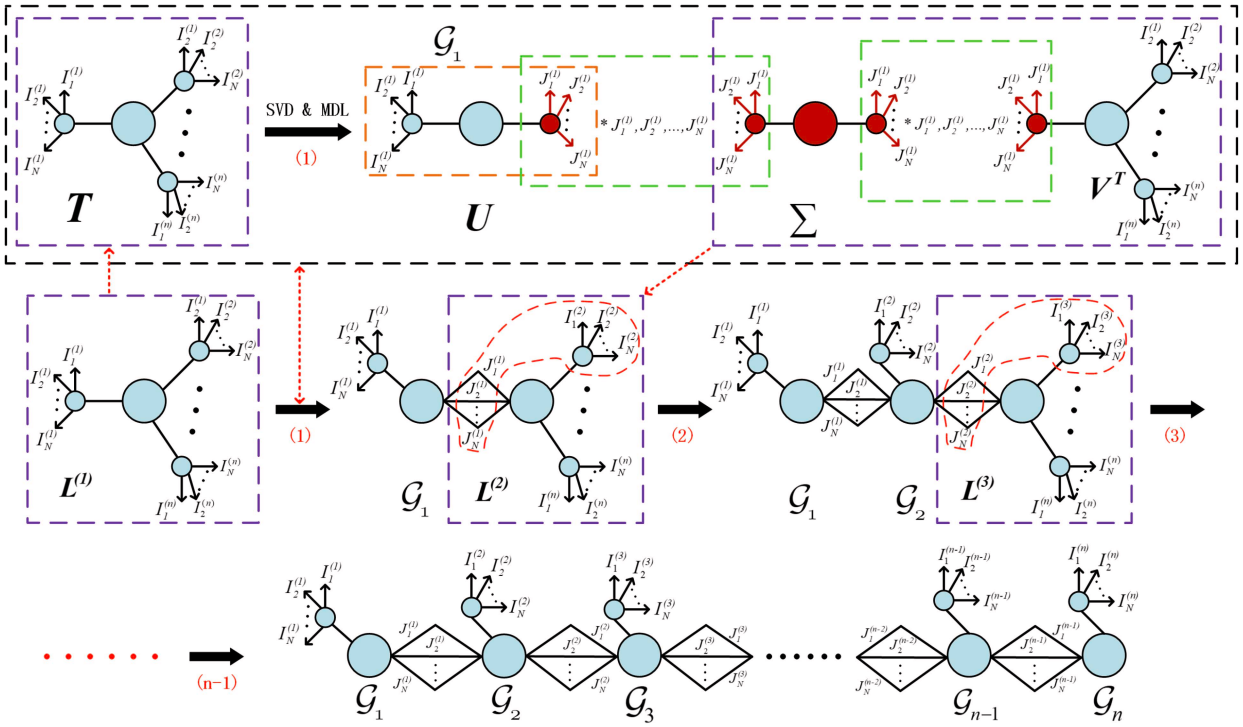


Fig. 8. The process of Improved and Efficient Tensor Train Decomposition(IETT). Firstly, estimating the TT rank of the tensor and applying IETT on each modal $I_1, I_2, \ldots, I_N$. Secondly, getting the reuse tensor for the next round of IETT. Finally, obtaining the core tensors: $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_N$.

where $\mathcal{S}_{(1)} \in \mathbb{R}^{L_1^{(1)} \times L_2^{(1)} \times \cdots \times L_N^{(1)} \times \cdots \times L_1^{(n)} \times L_2^{(n)} \times \cdots \times L_N^{(n)}}$ represents the diagonal tensor, $\mathcal{V}_{(1)}^T \in \mathbb{R}^{L_1^{(2)} \times L_2^{(2)} \times \cdots \times L_N^{(2)} \times \cdots \times L_1^{(n)} \times L_2^{(n)} \times \cdots \times L_N^{(n)}}$ and $\mathcal{U}_{(1)} \in \mathbb{R}^{L_1^{(1)} \times L_2^{(1)} \times \cdots \times L_N^{(1)}}$ denote the orthogonal tensors. Thus, the core tensor $\mathcal{G}_1$ is able to acquire,

$$\mathcal{G}_1 = \mathcal{U}_{(1)}. \tag{15}$$

In the subsequent decomposition, tensor $\mathcal{X}_2$ is derived from the product of tensors $\mathcal{S}_{(1)}$, $\mathcal{V}_{(1)}^T$,

$$\mathcal{X}_2 = \mathcal{S}_{(1)} * \mathcal{V}_{(1)}^T. \tag{16}$$

Hence, we can decompose $\mathcal{X}_2$ on dimension $L_1^{(2)}, L_2^{(2)}, \ldots, L_N^{(2)}$,

$$\mathcal{X}_2 = \mathcal{U}_{(2)} * \mathcal{S}_{(2)} * \mathcal{V}_{(2)}^T, \tag{17}$$

where $\mathcal{S}_{(2)} \in \mathbb{R}^{L_1^{(1)} L_1^{(2)} \times L_2^{(1)} L_2^{(2)} \times \cdots \times L_N^{(1)} L_N^{(2)} \times L_1^{(3)} L_2^{(3)} \times \cdots \times L_N^{(3)} \times \cdots \times L_1^{(n)} \times L_2^{(n)} \times \cdots \times L_N^{(n)}}$ represents the diagonal tensor, $\mathcal{V}_{(2)}^T \in \mathbb{R}^{L_1^{(3)} \times L_2^{(3)} \times \cdots \times L_N^{(3)} \times \cdots \times L_1^{(n)} \times L_2^{(n)} \times \cdots \times L_N^{(n)}}$ and $\mathcal{U}_{(2)} \in \mathbb{R}^{L_1^{(1)} L_1^{(2)} \times L_2^{(1)} L_2^{(2)} \times \cdots \times L_N^{(1)} L_N^{(2)}}$ denote the orthogonal tensors. Thus, the core tensor $\mathcal{G}_2$ is able to acquire,

$$\mathcal{G}_2 = \mathcal{U}_{(2)}. \tag{18}$$

In a similar way, tensor $\mathcal{X}_3$ is derived from the product of tensors $\mathcal{S}_{(2)}$, $\mathcal{V}_{(2)}^T$,

$$\mathcal{X}_3 = \mathcal{S}_{(2)} * \mathcal{V}_{(2)}^T. \tag{19}$$

Hence, we can decompose $\mathcal{X}_3$ on dimension $L_1^{(3)}, L_2^{(3)}, \ldots, L_N^{(3)}$,

$$\begin{aligned} \mathcal{X}_3 &= \mathcal{U}_{(3)} * \mathcal{S}_{(3)} * \mathcal{V}_{(3)}^T. \\ &, \ldots \\ &, \ldots \\ \mathcal{X}_{(n-1)} &= \mathcal{U}_{(n-1)} * \mathcal{S}_{(n-1)} * \mathcal{V}_{(n-1)}^T. \\ \mathcal{G}_{n-1} &= \mathcal{U}_{(n-1)}. \end{aligned} \tag{20}$$

Ultimately, after n iterations of decomposition, tensor $\mathcal{X}_n$ is derived from the product of tensors $\mathcal{S}_{(n-1)}$, $\mathcal{V}_{(n-1)}^T$,

$$\mathcal{X}_n = \mathcal{S}_{(n-1)} * \mathcal{V}_{(n-1)}^T, \tag{21}$$

where $\mathcal{S}_{(n-1)} \in \mathbb{R}^{L_1^{(1)} L_1^{(2)} \cdots L_1^{(n-1)} \times L_2^{(1)} L_2^{(2)} \cdots L_2^{(n-1)} \times \cdots \times L_N^{(1)} L_N^{(2)} \cdots L_N^{(n-1)} \times L_1^{(n)} L_2^{(n)} \cdots L_N^{(n)}}$ represents the diagonal tensor, $\mathcal{V}_{(2)}^T \in \mathbb{R}^{L_1^{(n)} \times L_2^{(n)} \times \cdots \times L_N^{(n)}}$ and $\mathcal{U}_{(2)} \in \mathbb{R}^{L_1^{(1)} L_1^{(2)} \cdots L_1^{(n-1)} \times L_2^{(1)} L_2^{(2)} \cdots L_2^{(n-1)} \times \cdots \times L_N^{(1)} L_N^{(2)} \cdots L_N^{(n-1)}}$ denote orthogonal tensors. Thus, the last core tensor $\mathcal{G}_n$ can be acquired,

$$\mathcal{G}_n = \mathcal{X}_{(n)}. \tag{22}$$

According to Fig. 4, equation (13) to equation (22) and related knowledge in Section III, we propose an improved and efficient TT decomposition algorithm(IETT) by combining low TT ranks based on MDL principle and the tensor reuse method as shown in Algorithm 1 and Fig. 8.

Algorithm 1 Algorithm of IETT

Input: An $(N \times n)$-dimensional tensor $\mathcal{X} \in \mathbb{R}^{L_1^{(1)} \times L_2^{(1)} \times \cdots \times L_N^{(1)} \times L_1^{(2)} \times L_2^{(2)} \times \cdots \times L_N^{(2)} \times \cdots \times L_1^{(n)} \times L_2^{(n)} \times \cdots \times I_N^{(n)}}$;
Accuracy of the decomposition ε.
Output: A series of TT cores $\mathcal{G}_1, \mathcal{G}_2, \cdots, \mathcal{G}_N$ that satisfied $||\mathcal{G}_j - \mathcal{H}_j||_F \leq \varepsilon ||\mathcal{G}_j||_F$, where $\mathcal{H}_j$ is an approximate of $\mathcal{G}_j$ and $j \in 1, 2, \ldots, N$.
1: Initialize $\mathcal{M}_1 = \mathcal{X}$, $R_0 = 1$;
2: **for** $i = 1$ to $n - 1$ **do**
3: Estimate the TT rank according to equation (13): $R_i = MDL(M_i)$
4: Compute R_i-Truncated SVD according to *Fig.*8: $M_i = \mathcal{U}_{(i)} \mathcal{S}_{(i)} \mathcal{V}_{(i)}^T + E_{(i)}$, where $||E_{(i)}||_F \leq \frac{\varepsilon}{\sqrt{n-1}} ||X||_F$
5: Compute the TT core according to equation (14) to equation (22): $\mathcal{G}_i = reshape(\mathcal{U}_{(i)}, [R_{i-1}, L_i, R_i])$
6: Compute $\hat{M} = S_{(i)} V_{(i)}^T$
7: Update the new tensor: $\mathcal{M}_{i+1} = reshape(\hat{M}, [R_{i-1} L_i, L_{i+1}, \cdots, L_n])$
8: **end for**
9: $\mathcal{G}_n = \mathcal{M}_n$
10: Return $\mathcal{H}$ with new TT cores: $\mathcal{G}_1, \mathcal{G}_2, \cdots, \mathcal{G}_n$

3) Large-Scale Information Processing Denoising Algorithm: Combining tensor smooth and reconstruction with IETT, a large-scale information processing denoising (LIPD) algorithm is presented in Algorithm 2, the elaboration of this process is shown as follows:

- *Algorithm 2, Line 1:*
  Datasets $\mathbf{X} \in \mathbb{R}^{M \times N}$ is folded into a $(K+1)$-dimensional tensor $\mathcal{X} \in \mathbb{R}^{M \times N_1 \times \cdots \times N_K}$, where $N_1 \times \cdots \times N_K = N$;
- *Algorithm 2, Line 4 $\sim$ 6:*
  According to equation (8), tensor $\mathcal{X}_{[m,:,\ldots,:]} \in \mathbb{R}^{N_1 \times \cdots \times N_K}$ is smoothed in the $k-th$ dimension, and L_k subtensors $\mathcal{X}_m^{(l_k)} \in \mathbb{R}^{N_k^{sub} \times \prod_{i \neq k} N_i}$ are obtained;
- *Algorithm 2, Line 7:*
  According to equation (9), the smoothing tensor $\mathcal{X}_{m,SS,k}^{(L_k)} \in \mathbb{R}^{N_k^{sub} \times L_k \times \prod_{i \neq k} N_i}$ can be obtained;
- *Algorithm 2, Line 8:*
  Smoothing tensor $\mathcal{X}_{m,SS,k}^{(L_k)}$ is denoised by IETT, and the calculating method is given in Algorithm 1;
- *Algorithm 2, Line 9:*
  According to equation (10) and equation (11), denoising tensor $\hat{\mathcal{X}}_{m,SS,k}^{(L_k)}$ is used to reconstruct and update tensor $\mathcal{X}_{[m,:,\ldots,:]}$. Then, $\mathcal{X}_{[m,:,\ldots,:]}$ is denoised in the dimension of $(k+1)-th$.
  Lines 4 $\sim$ 9 will repeat until all dimensions are updated.
- *Algorithm 2, Line 11:*
  When the loop is finished, tensor $\mathcal{X}_{[m,:,\ldots,:]}$ is denoised by IETT for the last time.
  Lines 4 $\sim$ 11 will iterate until the denoising of the entire datasets is completed.
- *Algorithm 2, Line 13:*
  Finally, the denoised tensor $\hat{\mathcal{X}} \in \mathbb{R}^{M \times N_1 \times \cdots \times N_K}$ is unfolded into the matrix $\hat{\mathbf{X}} \in \mathbb{R}^{M \times N}$.

Algorithm 2 Algorithm of LIPD

Input: Matrix $\mathbf{X} \in \mathbb{R}^{M \times N}$;
Max number of subarrays $L_k \in \{L_1, \ldots, L_K\}$.
Output: Denoised matrix $\hat{\mathbf{X}} \in \mathbb{R}^{M \times N}$.
1: Fold matrix $\mathbf{X} \in \mathbb{R}^{M \times N}$ into tensor $\mathcal{X} \in \mathbb{R}^{M \times N_1 \times \cdots \times N_K}$;
2: **for** $m = 1$ to M **do**
3: **for** $k = 1$ to K **do**
4: **for** $l_k = 0$ to L_k **do**
5: $\mathcal{X}_m^{(l_k)} = \mathcal{X}_{[m,:,\ldots,l_k:l_k+N_k^{sub},\ldots,:]}$
6: **end for**
7: $\mathcal{X}_{m,SS,k}^{(L_k)} = [\mathcal{X}_m^{(1)}, \cdots, \mathcal{X}_m^{(L_k)}]$
8: $\mathcal{X}_{m,SS,k}^{(L_k)} \xrightarrow{IETT} \hat{\mathcal{X}}_{m,SS,k}^{(L_k)}$
9: Update $\mathcal{X}_{[m,:,\ldots,:]}$ by $\hat{\mathcal{X}}_{m,SS,k}^{(L_k)}$
10: **end for**
11: $\mathcal{X}_{[m,:,\ldots,:]} \xrightarrow{IETT} \hat{\mathcal{X}}_{[m,:,\ldots,:]}$
12: **end for**
13: Unfold tensor $\hat{\mathcal{X}} \in \mathbb{R}^{M \times N_1 \times \cdots \times N_K}$ into matrix $\hat{\mathbf{X}} \in \mathbb{R}^{M \times N}$.

After preprocessing and denoising, the noise of datasets is extremely attenuated, which is of great help to classify.

C. Data Classification

LightGBM [23] is an efficient gradient boosting decision tree (GBDT) based framework that enables high accuracy, low memory consumption and fast speed through distributed computing when classifying large-scale datasets. These characteristics, especially in terms of memory advantages, fit seamlessly with IETT and have become significant reasons for choosing LightGBM.

V. Case Study

In this section, the proposed framework BDIP will be applied to large-scale DDoS attack detection scenarios and verified by a series of experiments. Section V-A describes the background of DDoS attack detection, and datasets used in experiments is illustrated in Section V-B. Section V-C lists experimental evaluation metrics, and the last subsection introduces the contrast experiments and results analysis.

A. Background Description

We applied BDIP to DDoS attack detection to verify the effectiveness of the framework. Fig. 9 shows a typical DDoS attack scenario where devices including large servers, personal computers, etc. are attacked as puppet machines (PM) to further attack infrastructures that connected to the Internet. PM refer to the compromised devices controlled by attackers. Once that happens, attackers will instruct PM to send massive attack packets to critical network components, resulting in initial partial denial of service and finally complete denial of service, which can make a huge trouble on information

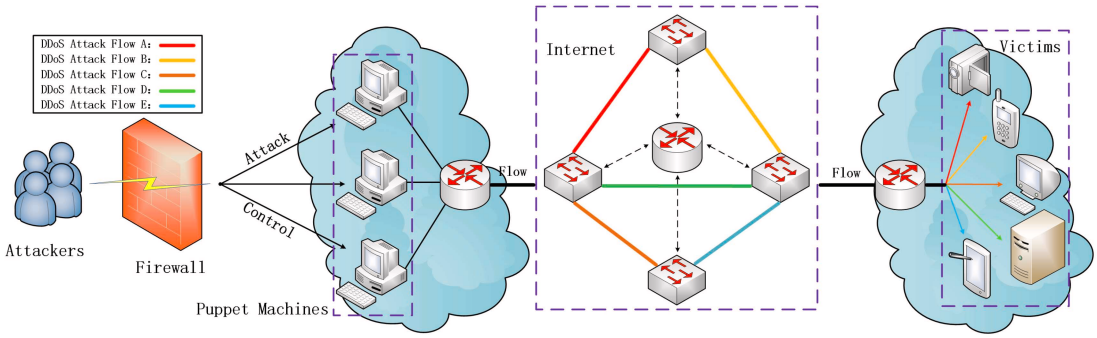


Fig. 9. A typical DDoS attack scenario.

processing. Therefore, it is meaningful to detect DDoS attacks in large-scale network timely.

B. DDoS Attack Datasets

To validate the experimental results, we employ the recently established datasets CIC-DDoS2019 and NLS-KDD for assessing performance metrics.

1) CIC-DDoS2019: CIC-DDoS2019 [24], which was developed by Cyber Security Research Laboratory of Concordia University, contains millions of network traffic data with 87 features. However, before the experiment, we will delete features that have little contribution to classification, including IP information, protocol type, time stamp and irrelevant identifier. After that, we will get 64 features shown in Table II. In order to express the feature name conveniently, we adopt the following abbreviations: FC symbolizes Flag-Count, PL symbolizes Packet-Length, SB symbolizes Subflow-Bwd, SF symbolizes Subflow-Fwd, TL symbolizes Total-Length, FPL, BPL symbolize Fwd-Packet-Length, Bwd-Packet-Length and FlI, FwI, BI symbolize Flow-IAT, Fwd-IAT, Bwd-IAT.

In addition, 40,000 traffic will be randomly selected from CIC-DDoS2019 as the experimental datasets. Considering that DDoS attacks do not occur as frequently as normal traffic, we will select 8,000 abnormal traffic and 32,000 normal traffic. More details are shown in Table III.

2) NSL-KDD: NLS-KDD [25] is also a widely used dataset in DDoS attack detection. Similar to CIC-DDoS2019, we delete 5 of 41 features before using it, and for the sake of avoiding overfitting, we increase the proportion of abnormal traffic to 40% while randomly selecting. Specific instructions are given in Table IV, Table V and we use DH to represent Dist-Host.

3) Feature Grouping: To optimize the utilization of tensors, we need to group the features extracted from the two datasets before experiment. To be specific, we try to put as many highly correlated features in the same small tensor as possible, so that they can be denoised and classified on the same computing core. For example, when processing CIC-DDoS2019, we are supposed to place the features *"Fwd-Packet-Length-Max, Fwd-Packet-Length-Min, Fwd-Packet-Length-Avg, Fwd-Packet-Length-Std-Dev"* and *"Bwd-Packet-Length-Max, Bwd-Packet-Length-Min, Bwd-Packet-Length-Avg, Bwd-Packet-Length-Std-Dev"* in two different tensors, because the former set of features pertains to *"Forward Traffic"*, while the latter set pertains to *"Backward Traffic"*.

TABLE II
FEATURES UTILIZED IN CIC-DDOS2019

	Name of features		Name of features
(1)	Source-Port	(33)	PL-Min
(2)	Destination-Port	(34)	PL-Max
(3)	Flow-Duration	(35)	PL-Avg
(4)	Total-Fwd-Packet	(36)	PL-Std-Dev
(5)	Total-Bwd-Packet	(37)	PL-Var
(6)	TL-Fwd-Packet	(38)	FIN-FC
(7)	TL-Bwd-Packet	(39)	SYN-FC
(8)	FPL-Max	(40)	RST-FC
(9)	FPL-Min	(41)	PUSH-FC
(10)	FPL-Avg	(42)	ACK-FC
(11)	FPL-Std-Dev	(43)	URG-FC
(12)	BPL-Max	(44)	CWE-FC
(13)	BPL-Min	(45)	ECE-FC
(14)	BPL-Avg	(46)	Fwd-Packet/s
(15)	BPL-Std-Dev	(47)	Avg-Packet-Size
(16)	Flow-Bytes/s	(48)	Avg-Fwd-SS
(17)	Flow-Packets/s	(49)	Avg-Bwd-SS
(18)	FlI-Avg	(50)	SF-Packets
(19)	FlI-Max	(51)	SF-Bytes
(20)	FlI-Min	(52)	SB-Packets
(21)	FwI-Total	(53)	SB-Bytes
(22)	FwI-Avg	(54)	Fwd-WB
(23)	FwI-Max	(55)	Bwd-WB
(24)	FwI-Min	(56)	Fwd-Active-DP
(25)	BI-Total	(57)	Fwd-Min-SS
(26)	BI-Avg	(58)	Avg-TAF
(27)	BI-Max	(59)	Bwd-Packet/s
(28)	BI-Min	(60)	Max-TAF
(29)	Fwd-HL	(61)	Min-TAF
(30)	Bwd-HL	(62)	Avg-TIF
(31)	Download/Upload-Ratio	(63)	Std-Dev-TIF
(32)	Std-Dev-TAF	(64)	Min-TIF

C. Evaluation Metrics

Some metrics will be introduced in this subsection. They are considered as macro-averaged, so they all have values between 0 and 1. In addition, we use TP, FN, FP, TN represents True Positive, False Negative, False Positive and True Negative, respectively.

1) The equation of Accuracy (Acc) is as follows:

$$Acc = \frac{TP + TN}{TP + FN + FP + TN}. \tag{23}$$

TABLE III
THE TYPES OF TRAFFIC AND THE NUMBER OF INSTANCES USED IN CIC-DDOS2019

Traffic Type	Traffic name	Number	Total
Normal	BENIGN	32,000	32,000
Abnormal	DNS	800	8,000
	LDAP	800	
	MSSQL	800	
	NetBIOS	800	
	NTP	800	
	SNMP	800	
	SSDP	800	
	UDP	800	
	SYN	800	
	TFTP	800	

TABLE IV
FEATURES UTILIZED IN NSL-KDD

	Name of features		Name of features
(1)	Duration	(19)	Srv-Count
(2)	Src-Bytes	(20)	Serror-Rate
(3)	Dst-Bytes	(21)	Srv-Serror-Rate
(4)	Land	(22)	Rerror-Rate
(5)	Wrong-Fragment	(23)	Srv-Rerror-Rate
(6)	Urgent	(24)	Same-Srv-Rate
(7)	Hot	(25)	Diff-Srv-Rate
(8)	Num-Failed-Logins	(26)	Srv-Diff-Host-Rate
(9)	Logged-In	(27)	DH-Count
(10)	Num-Compromised	(28)	DH-Srv-Count
(11)	Root-Shell	(29)	DH-Same-Srv-Rate
(12)	Su-Attempted	(30)	DH-Diff-Srv-Rate
(13)	Bum-Root	(31)	DH-Same-Src-Port-Rate
(14)	Num-File-Creations	(32)	DH-Rerror-Rate
(15)	Num-Shells	(33)	DH-Serror-Rate
(16)	Num-Access-Files	(34)	DH-Srv-Serror-Rate
(17)	Is-Host-Login	(35)	DH-Rerror-Rate
(18)	Is-Guest-Login	(36)	DH-Srv-Rerror-Rate

2) The equation of Precision (Pre) is as follows:

$$Pre = \frac{TP}{TP + FP}. \tag{24}$$

3) The equation of Recall (Rec) is as follows:

$$Rec = \frac{TP}{TP + FN}. \tag{25}$$

4) The equation of F1-Score is as follows::

$$F1 - Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}. \tag{26}$$

D. Contrast Experiments

In this subsection, a group of contrast experiments are carried out to evaluate the performance of BDIP under different conditions. Firstly, we determine the experimental parameters and explain their rationality. Secondly, we consider the performance under different signal-to-noise ratio (SNR). After that, we give the classification results based on different denoising algorithms. Then, we show the detection results based on different classification algorithms. Finally, we asses the comprehensive performance against other frameworks.

TABLE V
THE TYPES OF TRAFFIC AND THE NUMBER OF INSTANCES USED IN NLS-KDD

Traffic Type	Traffic name	Number	Total
Normal	Legitimate	50,000	50,000
Abnormal	Apache2	730	66,370
	Back	1,300	
	IpSweep	3,740	
	Satan	4,360	
	Neptune	45,800	
	WarezClient	890	
	Nmap	1,560	
	Smurf	3,300	
	GuessPassword	1,610	
	PortSweep	3,080	

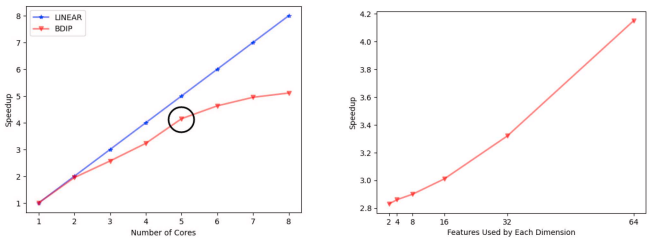
1) Determination of Experimental Parameters: In order to achieve the optimal detection results, we need to set the suitable parameters before experiments. Firstly, for initial datasets CIC-DDoS2019 and NSL-KDD, they will be expressed as $\mathbf{X} \in \mathbb{R}^{M \times N}$, where M is the total number of data, N is the feature number of datasets. And in the algorithm of LIPD, both datasets will be folded into the three-dimensional tensor $\mathcal{X} \in \mathbb{R}^{M \times N_1 \times N_2}$, in which $N = N_1 \times N_2$, $N_1 = N_2 = 8$ of CIC-DDoS2019, $N_1 = N_2 = 6$ of NSL-KDD. Secondly, 5-fold cross validation is used to make training sets and testing sets. Finally, experiments are conducted in the distributed parallel environment. Thus, a suitable speedup needs to be selected. As the key performance indicator in parallel computing, speedup is calculated by dividing the execution time of a sequential program by that of its parallel version. The formula for this assessment is presented below:

$$S = \frac{T_s}{T_p}, \tag{27}$$

where S represents speedup, execution times for the serial and parallel versions of a program are denoted by T_s and T_p.

Therefore, the increase in the number of computing cores can decrease the computing time and enhance the speedup, but it may also raise other costs such as the communication between computing cores and synchronization wait. Hence, the speedup curve does not increase linearly, its growth rate slow down gradually and will appear the inflection point. In order for our model to be widely used, especially for individual users who do not have many computing resources, we need to choose an optimal speedup. Fig. 10(a) illustrates how the speedup of BDIP varies with the number of computing cores. We can find that speedup increases very fast when the number of cores is less than 5, and becomes very slow when the cores is greater than 5. Overall consideration, we choose the speedup of 5 to balance computation cost and speed.

Fig. 11(b) explains this in more detail. We observe how speedup changes by adjusting the tensor scale of the input



(a) (b)

Fig. 10. The choice of Speedup: (a) Speedup varies with the number of computing cores. (b) Speedup varies with the number of features used in each dimension.

datasets. Taking CIC-DDoS2019 as an example, it is first represented as a set of four-dimensional tensors with 64 traffic data of each dimension. Then, we adjust the scale of each dimension, and finally use the BDIP framework to process it in parallel with 5 computing cores. Results shown in Fig. 10(b) indicates that as the data scale increases of each dimension, the speedup also increases. In conclusion, BDIP is naturally a DDoS attack detection framework suitable for big data.

2) Different SNR: As the initial level of noise in datasets is unknown, it can first be assumed as noise-free [4]. Then zero-mean white Gaussian noise is introduced into datasets through the supervised manner. This allows us to assess the performance of frameworks across a range of $-10dB$ to $10dB$ by evaluating SNR_{out}. The equation is as follows,

$$SNR_{out} = 10 \cdot log_{10}\left(\frac{\|\mathcal{X}_2\|_F^2}{\|\mathcal{X}_2 - \mathcal{X}_0\|_F^2}\right)(dB), \tag{28}$$

where $\mathcal{X}_2$ denotes the denoised data, $|\cdot|_F$ represents the Frobenius norm.

In order to ensure the effectiveness, we have chosen a mix of established and contemporary frameworks for comparison with BDIP on CIC-DDoS2019. Specifically, we include the traditional methods of SVD [26] and HOSVD [27], alongside the more recent MUDE [21] and TSSD [28], which have demonstrated promising outcomes. In addition, we choose $L_1 = L_2 = 2$ in Algorithm 2. The result is shown in Fig. 11, it can be found that $SNR_{in} \approx SNR_{out}$ without any denoising. Moreover, all denoised datasets are superior to the original one, and HOSVD should perform better than SVD, because SVD processes data from the matrix perspective but HOSVD uses tensor structures to perform SVD on higher dimension. Furthermore, HOSVD and SVD perform denoising across the entire dataset $\mathcal{X} \in \mathbb{R}^{M \times N}$, while recently proposed MUDE, TSSD and BDIP target the denoising of individual data instances $\mathcal{X}_{[m,:,:]} \in \mathbb{R}^{N_1 \times N_2}$, which can reduce the noise more effectively. Experiments demonstrate BDIP outperforms other methods by efficiently preserving data characteristics and capitalizing on the strengths of tensor.

3) Different Denoising Algorithm: Our assessment of LIPD's denoising efficacy is conducted by comparing it with SVD, HOSVD, MUDE, and TSSD under uniform SNR conditions set at $0dB$, with LightGBM serving as the classification algorithm. Experiments are visualized in Fig. 12 and Fig. 13, which depict the fluctuation of evaluation metrics

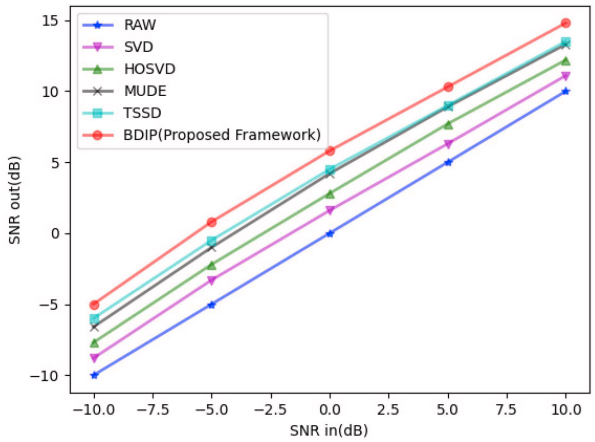
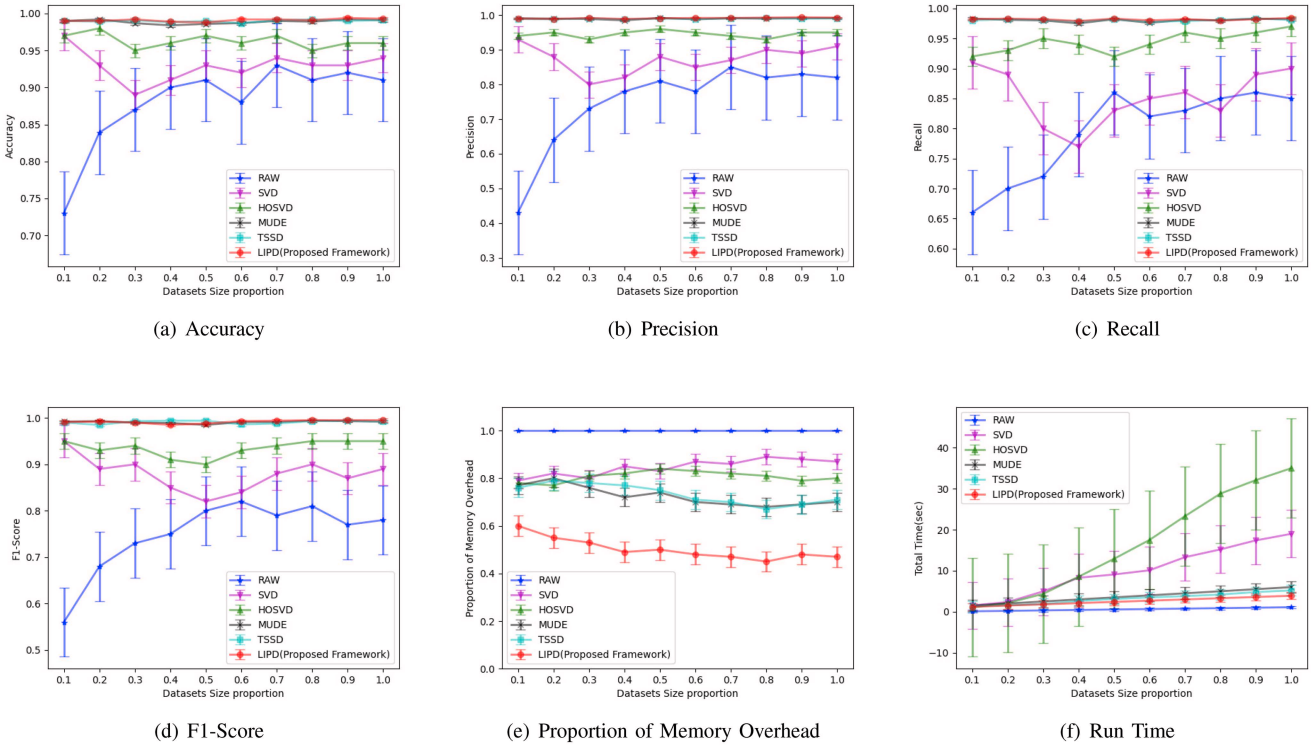


Fig. 11. SNR_{out} with SNR_{in} for tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times N_2 \times M}$.

corresponding to the CIC-DDoS2019 and NSL-KDD datasets, respectively. It is observed that the performance of all frameworks improves with an increasing proportion of the datasets, as this allows for a richer set of features to be presented to the classification algorithm. Additionally, HOSVD generally outperforms SVD across most scenarios, due to its more pronounced denoising capabilities in higher dimensions and its ability to conduct multiple SVDs simultaneously, thereby compensating for the shortcomings of machine learning classifiers. LIPD, in most evaluation metrics, is either slightly superior or comparable to MUDE and TSSD, with the exception of memory consumption. This is attributed to the significant advantages of IETT-based frameworks over SVD-based ones in high-dimensional contexts, which is crucial for detecting DDoS attacks in large-scale and heterogeneous networks.

4) Different Classification Algorithm: We evaluate the impact of various classification algorithms on experimental outcomes, with all tests performed at an SNR level of $0dB$ and denoising carried out via the IETT method. In this comparison, we analyze the performance of LightGBM in contrast to Logistic Regression (LR), SVM (Support Vector Machine), MLP (Multi Layer Perceptron), XGBoost [29], and CatBoost [30] on two datasets. The findings, depicted in Fig. 14 and Fig. 15, indicate that LR generally delivers the least favorable results across all metrics, despite its quick execution time. Conversely, LightGBM exhibits the most impressive performance, particularly with extensive datasets, aligning perfectly with the IETT framework.

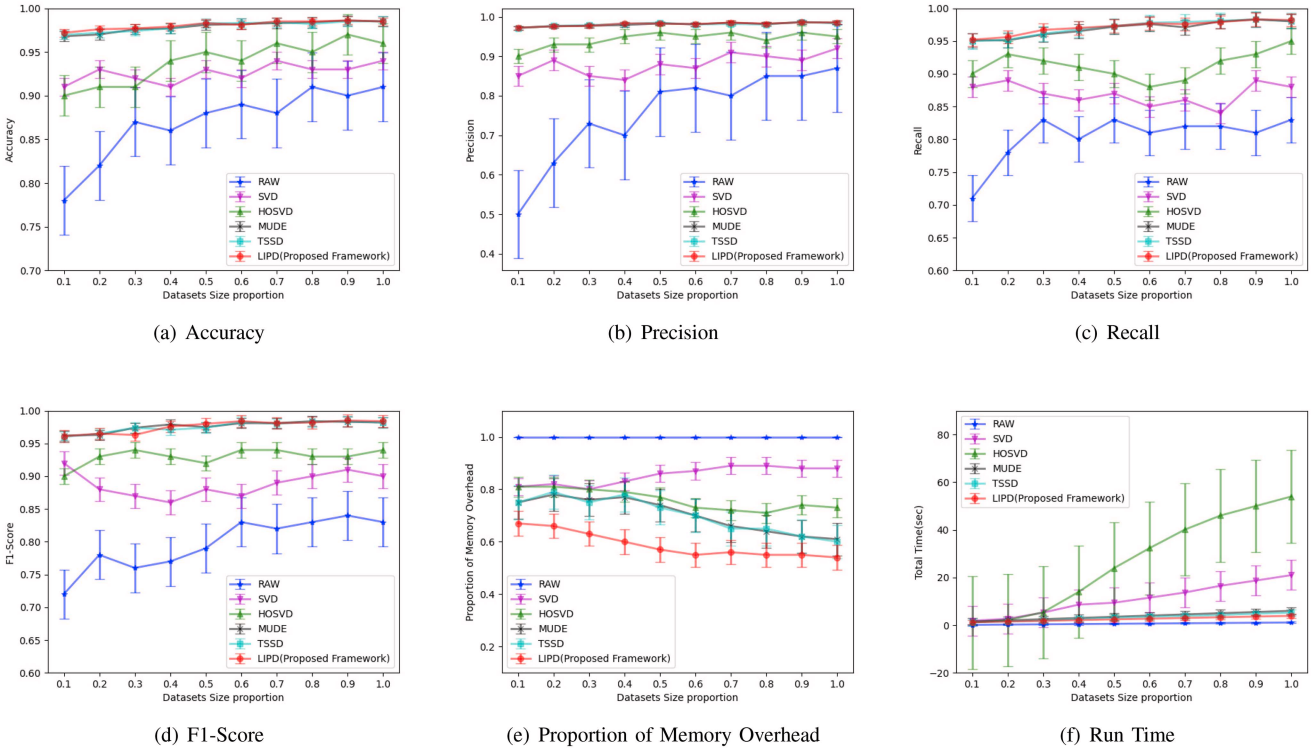
5) Comparison With Related Frameworks: Lastly, we have chosen a number of pertinent frameworks for a comparative evaluation against BDIP, focusing on their detection capabilities on NSL-KDD. To align with the prevailing practices where the majority of related studies utilize noise-free datasets, we have set the SNR at a high level of $35dB$ to replicate a low-noise environment. The comparative results are detailed in Table VI. Our proposed BDIP framework outperforms the others in terms of accuracy. While TBF may show the highest *F1-Score*, it comes with a significantly higher false alarm rate. MMNR delivers a performance close to our method, if the consumption of memory is not a critical factor, owing to its tensor-based approach similar to ours.



(a) Accuracy (b) Precision (c) Recall

(d) F1-Score (e) Proportion of Memory Overhead (f) Run Time

Fig. 12. Performance Comparison of Different Denoising Algorithms on CIC-DDoS2019.



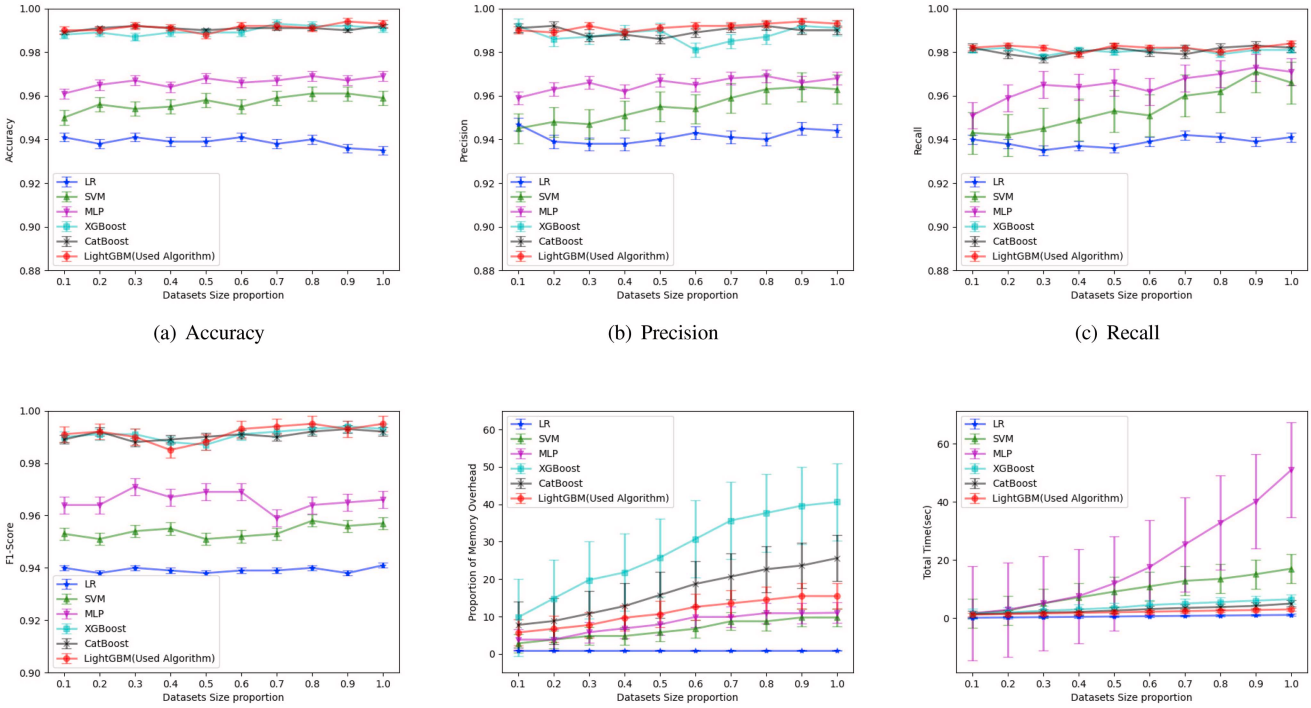
(a) Accuracy (b) Precision (c) Recall

(d) F1-Score (e) Proportion of Memory Overhead (f) Run Time

Fig. 13. Performance Comparison of Different Denoising Algorithms on NSL-KDD.

VI. Conclusion

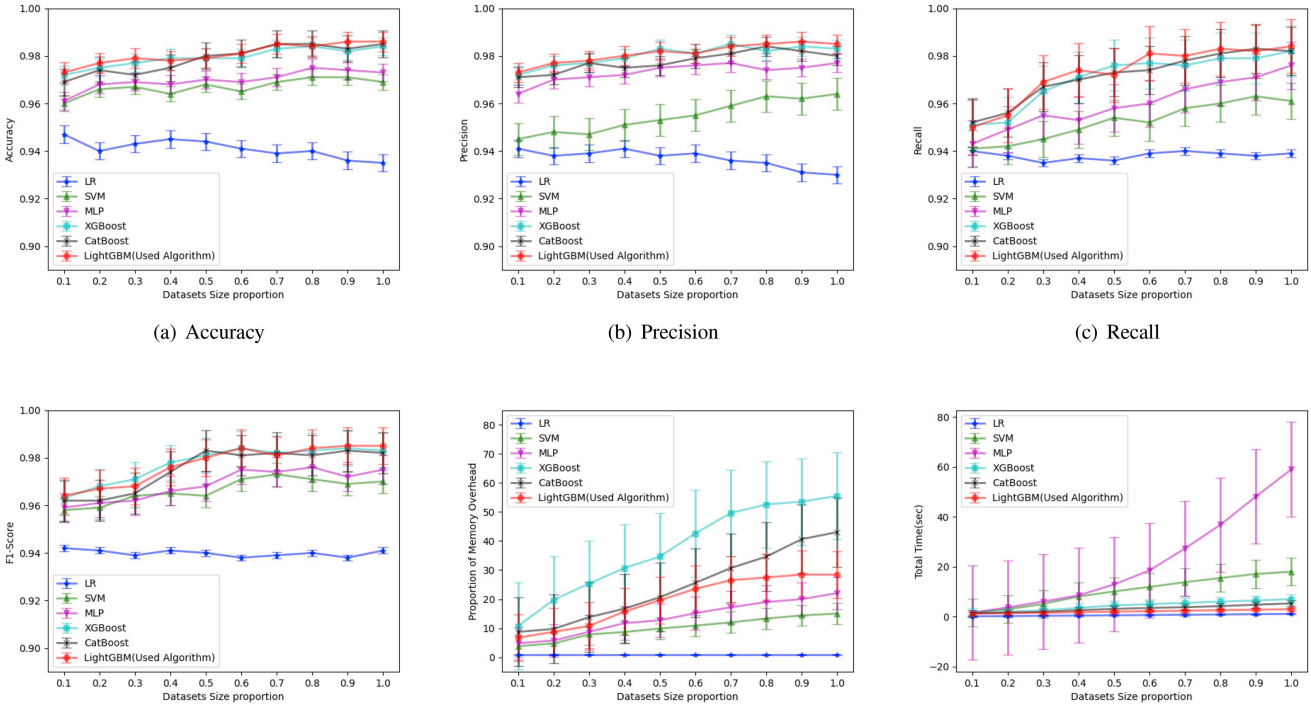
In this study, we introduce an efficient information processing framework known as BDIP, which is designed to handle big data challenges and has been implemented for the detection of Distributed Denial of Service attacks. The framework is comprised of three main components: preprocessing, denoising, classification. Highlight is the proposal of IETT and LIPD. They can effectively denoise through smooth reconstruction of tensor space, which not only provides clean datasets for LightGBM to classify, but also enhances the computing



(a) Accuracy (b) Precision (c) Recall

(d) F1-Score (e) Proportion of Memory Overhead (f) Run Time

Fig. 14. Performance Comparison of Different Classification Algorithms on CIC-DDoS2019.



(a) Accuracy (b) Precision (c) Recall

(d) F1-Score (e) Proportion of Memory Overhead (f) Run Time

Fig. 15. Performance Comparison of Different Classification Algorithms on NSL-KDD.

power and greatly reduces the consumption of storage. The subsequent contrast experiments have proved its effectiveness from several aspects. Compared with previous works, the tensor-based framework can better process large-scale high-dimensional information, while BDIP are naturally suitable for these scenarios, thus improving the detection accuracy and preventing the occurrence of dimensional disasters.

As for future work, we intend to do further research on distributed computing, a optimal speedup ratio can greatly improve the experimental efficiency. Additionally,

TABLE VI
COMPARISON OF DIFFERENT DDoS ATTACK DETECTION FRAMEWORKS

Model	Feature Number	Accuracy(%)	False Alarm Rate(%)	F1-Score(%)
BDIP(Proposed Framework)	36	**99.19**	**0.61**	99.10
MLP in [31]	41	96.30	5.00	97.10
CTSBS-MLP in [10]	41	97.61	0.63	96.82
SBS-MLP in [10]	31	97.66	0.62	96.87
MCLP in [32]	17	98.26	2.03	97.89
TBF in [21]	36	98.54	1.08	**99.28**
MMNR in [28]	36	98.84	0.82	98.74

auto-generating data-plane pipelines [33] are also worth considering. It offers network operators a straightforward means to convey their requirements to compilers, removing the manual task of fine-tuning hyperparameters and managing diverse network data plane and topological demands, thus enhancing the efficiency of intrusion detection.

REFERENCES

[1] B. Coelho and A. Schaeffer-Filho, "BACKORDERS: Using random forests to detect DDoS attacks in programmable data planes," in *Proc. 5th Int. Workshop P4 Europe*, 2022, pp. 1–7.

[2] M. Mittal, K. Kumar, and S. Behal, "DDoS-AT-2022: A distributed denial of service attack dataset for evaluating DDoS defense system," in *Proc. Indian Nat. Sci. Acad.*, 2023, pp. 1–19.

[3] P. Wang, L. T. Yang, J. Li, X. Li, and X. Zhou, "RM^{2}T^{2}C: Retrospective multivariate Multistep transition tensor chain model for user mobility pattern prediction," *IEEE Trans. Ind. Informat.*, vol. 18, no. 10, pp. 6991–6999, Oct. 2020.

[4] J. A. Sáez, M. Galar, J. Luengo, and F. Herrera, "Tackling the problem of classification with noisy data using multiple classifier systems: Analysis of the performance and robustness," *Inf. Sci.*, vol. 247, pp. 1–20, Jun. 2013.

[5] J. P. Anderson, *Computer Security Threat Monitoring and Surveillance, James P*, Anderson Co., Fort Washington, PA, USA, 1980.

[6] D. E. Denning, "An intrusion-detection model," *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[7] M. H. Bhuyan, A. Kalwar, A. Goswami, D. Bhattacharyya, and J. Kalita, "Low-rate and high-rate distributed DoS attack detection using partial rank correlation," in *Proc. 5th Int. Conf. Commun. Syst. Netw. Technol.*, 2015, pp. 706–710.

[8] S. D. Çakmakçı, T. Kemmerich, T. Ahmed, and N. Baykal, "Online DDoS attack detection using Mahalanobis distance and kernel-based learning algorithm," *J. Netw. Comput. Appl.*, vol. 168, Oct. 2020, Art. no. 102756.

[9] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. Ramos, and A. Madeira, "FlowLens: Enabling efficient flow classification for ML-based network security applications," in *Proc. NDSS*, 2021, pp. 1–18.

[10] M. Wang, Y. Lu, and J. Qin, "A dynamic MLP-based DDoS attack detection method using feature selection and feedback," *Comput. Security.*, vol. 88, Jan. 2020, Art. no. 101645.

[11] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martinez-del Rincon, and D. Siracusa, "LUCID: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 876–889, Jun. 2020.

[12] G. Apruzzese, P. Laskov, and J. Schneider, "SoK: Pragmatic assessment of machine learning for network intrusion detection," 2023, *arXiv:2305.00550*.

[13] A. B. Dehkordi, M. Soltanaghaei, and F. Z. Boroujeni, "The DDoS attacks detection through machine learning and statistical methods in SDN," *J. Supercomput.*, vol. 77, pp. 2383–2415, Mar. 2021.

[14] T. Swamy, A. Rucker, M. Shahbaz, I. Gaur, and K. Olukotun, "Taurus: A data plane architecture for per-packet ML," in *Proc. 27th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2022, pp. 1099–1114.

[15] M. Wichtlhuber et al., "IXP scrubber: Learning from blackholing traffic for ML-driven DDoS detection at scale," in *Proc. ACM SIGCOMM Conf.*, 2022, pp. 707–722.

[16] T. G. Kolda and B. W. Bader, "Tensor decompositions and applications," *SIAM Rev.*, vol. 51, no. 3, pp. 455–500, 2009.

[17] T.-J. Shan, M. Wax, and T. Kailath, "On spatial smoothing for direction-of-arrival estimation of coherent signals," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 33, no. 4, pp. 806–811, Aug. 1985.

[18] Q. Fan et al., "IDAD: An improved tensor train based distributed DDoS attack detection framework and its application in complex networks," *Future Gener. Comput. Syst.*, vol. 162, Jan. 2025, Art. no. 107471.

[19] I. V. Oseledets, "Tensor-train decomposition," *SIAM J. Sci. Comput.*, vol. 33, no. 5, pp. 2295–2317, 2011.

[20] R. Fan et al., "A novel multi-modal incremental tensor decomposition for anomaly detection in large-scale networks," *Inf. Sci.*, vol. 681, Oct. 2024, Art. no. 121210.

[21] J. P. A. Maranhão, J. P. C. da Costa, E. Javidi, C. A. B. de Andrade, and R. T. de Sousa Jr., "Tensor based framework for distributed denial of service attack detection," *J. Netw. Comput. Appl.*, vol. 174, Jan. 2021, Art. no. 102894.

[22] M. Wax and T. Kailath, "Detection of signals by information theoretic criteria," *IEEE Trans. Acoust., speech, signal Process.*, vol. 33, no. 2, pp. 387–392, Apr. 1985.

[23] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–9.

[24] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Security. Technol. (ICCST)*, 2019, pp. 1–8.

[25] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Security. Defense Appl.*, 2009, pp. 1–6.

[26] Q. Guo, C. Zhang, Y. Zhang, and H. Liu, "An efficient SVD-based method for image denoising," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 26, no. 5, pp. 868–880, May 2016.

[27] A. Rajwade, A. Rangarajan, and A. Banerjee, "Image denoising using the higher order singular value decomposition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 4, pp. 849–862, Apr. 2013.

[28] J. Xu, X. Li, P. Wang, X. Jin, and S. Yao, "Multi-modal noise-robust DDoS attack detection architecture in large-scale networks based on tensor SVD," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 152–165, Jan./Feb. 2023.

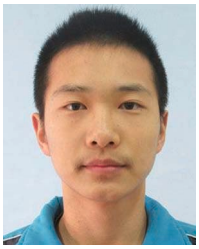
[29] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[30] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 6639–6649.

[31] G. S. Kushwah and S. T. Ali, "Detecting DDoS attacks in cloud computing using ANN and black hole optimization," in *Proc. 2nd Int. Conf. Telecommun. Netw. (TEL-NET)*, 2017, pp. 1–5.

[32] S. M. H. Bamakan, H. Wang, T. Yingjie, and Y. Shi, "An effective intrusion detection framework based on MCLP/SVM optimized by time-varying chaos particle swarm optimization," *Neurocomputing*, vol. 199, pp. 90–102, Jul. 2016.

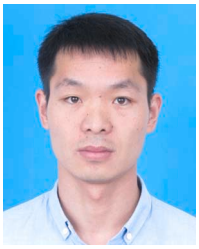
[33] T. Swamy, A. Zulfiqar, L. Nardi, M. Shahbaz, and K. Olukotun, "Homunculus: Auto-generating efficient data-plane ml pipelines for datacenter networks," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2023, pp. 329–342.



Qiyuan Fan received the B.E. degree in computer science and technology from the Wuhan University of Technology, Wuhan, China. He is currently pursuing the master's degree with the School of Software, Yunnan University, Kunming, China. His current research interests include big data, tensor network, and reinforcement learning.

Xue Li received the B.E. degree in electronic and information engineering from Zhengzhou University, Zhengzhou, China, the M.E. degree in control engineering from the Wuhan University of Technology, Wuhan, China. She is currently a Lecturer with the School of Electronic Information Engineering, Henan Institute of Technology, Xinxiang, China. Her research interests include big data and electronic information.

Puming Wang received the B.E. degree in communication engineering from Xidian University, Xi'an, China, the M.E. degree in control engineering from the Wuhan University of Technology, Wuhan, China, and the Ph.D. degree from the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan. He is currently an Associate Professor with the School of Software, Yunnan University, Kunming, China. His research interests include big data, artificial intelligence, and network security.



Xin Jin received the B.S. degree in electronics and information engineering from Henan Normal University, Xinxiang, China, and the Ph.D. degree in communication and information systems from Yunnan University, Kunming, China, where he is currently an Associate Professor with the School of Software. His research interests include pulse coupled neural networks theory and its applications, image processing, information fusion, optimization algorithm, and bio-informatics.

Shaowen Yao received the B.S. and M.S. degrees in telecommunication engineering from Yunnan University, Kunming, China, and the Ph.D. degree in computer application technology from the University of Electronic Science and Technology of China, Chengdu, China. He is currently a Professor with the School of Software, Yunnan University. His current research interests include neural network theory and applications, cloud computing, and big data computing.

Shengfa Miao (Member, IEEE) received the B.S. and M.S. degrees in computer science and technology from Lanzhou University, Lanzhou, China, and the Ph.D. degree in data mining from Leiden University, Leiden, The Netherlands. He is currently an Associate Professor with the School of Software, Yunnan University, Kunming, China. His current research interests include business risk control structural health monitoring and digital twin.

Sizhang Li received the B.E. degree from the Internet of Things Engineering, Hunan Institute of Engineering, Hengyang, China. She is currently pursuing the master's degree with the School of Software, Yunnan University, Kunming, China. Her current research interests include cloud computing and big data security.

Min An received the B.S. degree in computer science and technology from Northwest Minzu University, Lanzhou, China. He is currently pursuing the master's degree with the School of Software, Yunnan University, Kunming, China. His current research interests include reinforcement learning, time series forecasting, block chain technology, and federated learning.

Jing Xu received the B.E. degree in mechanical engineering from Hefei University, Hefei, China, and the M.S. degree from the School of Software, Yunnan University, Kunming, China. His current research interests include cloud computing and big data security.